

C# Alapvető Technikák Összefoglaló

1. ArgumentException használata

Mi az ArgumentException?

Az `ArgumentException` egy beépített kivétel típus, amelyet akkor dobunk, amikor egy metódus vagy property érvénytelen paramétert kap.

Alapvető használat

```
if (string.IsNullOrEmpty(value))
{
    throw new ArgumentException("A paraméter nem lehet üres!");
}

if (age < 0 || age > 150)
{
    throw new ArgumentException("Az életkor 0 és 150 között kell legyen!");
}
```

Részletes hibaüzenet megadása

```
// Egyszerű hibaüzenet
throw new ArgumentException("Hibás érték!");

// Hibaüzenet + paraméter neve
throw new ArgumentException("Az érték nem lehet negatív!", nameof(value));

// Hibaüzenet + paraméter név + belső kivétel
throw new ArgumentException("Hibás formátum!", nameof(email), innerException);
```

Property-kben való használat

```
private string _name;

public string Name
{
    get { return _name; }
    set
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("A név nem lehet üres!");

        if (value.Length < 2)
```

```
        throw new ArgumentException("A név legalább 2 karakter hosszú kell  
legyen!");  
  
        _name = value;  
    }  
}
```

Try-Catch használata ArgumentException-nel

```
try  
{  
    user.Name = ""; // Ez ArgumentException-t dob  
}  
catch (ArgumentException ex)  
{  
    Console.WriteLine($"Hibás adat: {ex.Message}");  
    // Vagy logging, vagy felhasználói üzenet megjelenítése  
}
```

2. DateTime számítások

Alapvető DateTime műveletek

Jelenlegi dátum és idő

```
DateTime now = DateTime.Now;           // Helyi idő  
DateTime utcNow = DateTime.UtcNow;     // UTC idő  
DateTime today = DateTime.Today;       // Ma, 00:00:00-kor
```

Dátum létrehozása

```
DateTime birthDate = new DateTime(1990, 5, 15);           // 1990. május 15.  
DateTime specificTime = new DateTime(2023, 12, 25, 14, 30, 0); // Idő megadással
```

Életkor számítása

```
public int CalculateAge(DateTime birthDate)  
{  
    DateTime today = DateTime.Now;  
    int age = today.Year - birthDate.Year;  
  
    // Ha még nem volt idén szülinap, egy évvel kevesebb
```

```
    if (today.Month < birthDate.Month ||  
        (today.Month == birthDate.Month && today.Day < birthDate.Day))  
    {  
        age--;  
    }  
  
    return age;  
}
```

Dátum összehasonlítás és validálás

```
// Jövőbeli dátum ellenőrzése  
if (birthDate > DateTime.Now)  
{  
    throw new ArgumentException("A születési dátum nem lehet jövőbeli!");  
}  
  
// Dátum tartomány ellenőrzése  
DateTime minDate = DateTime.Now.AddYears(-120);  
DateTime maxDate = DateTime.Now.AddYears(-13);  
  
if (birthDate < minDate || birthDate > maxDate)  
{  
    throw new ArgumentException("Az életkor 13 és 120 év között kell legyen!");  
}
```

Időszámítás

```
// Napok számítása két dátum között  
DateTime startDate = new DateTime(2023, 1, 1);  
DateTime endDate = DateTime.Now;  
int daysDifference = (endDate - startDate).Days;  
  
// TimeSpan használata  
TimeSpan difference = endDate - startDate;  
Console.WriteLine($"Napok: {difference.Days}");  
Console.WriteLine($"Órák: {difference.TotalHours}");  
Console.WriteLine($"Percek: {difference.TotalMinutes}");
```

Következő születésnap számítása

```
public int DaysUntilNextBirthday(DateTime birthDate)  
{  
    DateTime today = DateTime.Now;  
    DateTime nextBirthday = new DateTime(today.Year, birthDate.Month,  
        birthDate.Day);
```

```
// Ha már volt idén, akkor jövő évben
if (nextBirthday < today)
{
    nextBirthday = nextBirthday.AddYears(1);
}

return (nextBirthday - today).Days;
}
```

Dátum formázás

```
DateTime now = DateTime.Now;

// Előre definiált formátumok
string shortDate = now.ToShortDateString(); // "2023.12.25"
string longDate = now.ToLongDateString();   // "2023. december 25., hétfő"
string shortTime = now.ToShortTimeString(); // "14:30"

// Egyedi formátumok
string custom1 = now.ToString("yyyy-MM-dd"); // "2023-12-25"
string custom2 = now.ToString("dd/MM/yyyy HH:mm"); // "25/12/2023 14:30"
string custom3 = now.ToString("MMMM dd, yyyy"); // "december 25, 2023"
```

3. Alapvető Regex használat

Regex importálása

```
using System.Text.RegularExpressions;
```

Alapvető mintillesztés

```
string pattern = @"^\d{3}-\d{2}-\d{4}$"; // 123-45-6789 formátum
string input = "123-45-6789";

bool isMatch = Regex.IsMatch(input, pattern);
Console.WriteLine($"Illeszkedik: {isMatch}"); // True
```

Gyakori minták

Nevek validálása (betűk és szóközők)

```
string namePattern = @"^[a-zA-ZáéíóöőúűÁÉÍÓÖŐÚÚ\Œ]+$";

bool isValidName = Regex.IsMatch("Nagy János", namePattern); // True
bool isValidName = Regex.IsMatch("Nagy123", namePattern); // False
```

Email validálás (alapszintű)

```
string emailPattern = @"^[^@\\Œ]+@[^@\\Œ]+\\.[^@\\Œ]+$";

bool validEmail = Regex.IsMatch("test@example.com", emailPattern); // True
bool invalidEmail = Regex.IsMatch("test@", emailPattern); // False
```

Jelszó validálás (összetett)

```
// Legalább egy kisbetű
bool hasLowercase = Regex.IsMatch(password, @"(?=.*[a-z])");

// Legalább egy nagybetű
bool hasUppercase = Regex.IsMatch(password, @"(?=.*[A-Z])");

// Legalább egy szám
bool hasNumber = Regex.IsMatch(password, @"(?=.*\d)");

// Legalább egy speciális karakter
bool hasSpecial = Regex.IsMatch(password, @"(?=.*[!@#$%^&*])");

// Összes egyben
string strongPasswordPattern = @"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*]).{8,}$";
bool isStrongPassword = Regex.IsMatch(password, strongPasswordPattern);
```

Számok validálása

```
// Csak számok
string numberPattern = @"^\d+$";
bool isNumber = Regex.IsMatch("12345", numberPattern); // True

// Pozitív egész számok
string positiveIntPattern = @"^[1-9]\d*$";
bool isPositiveInt = Regex.IsMatch("123", positiveIntPattern); // True

// Tizedes számok
string decimalPattern = @"^\d+(\.\d{1,2})?";
bool isDecimal = Regex.IsMatch("123.45", decimalPattern); // True
```

Regex metódusok

IsMatch - egyezés ellenőrzése

```
bool matches = Regex.IsMatch(input, pattern);
```

Match - első egyezés megkeresése

```
Match match = Regex.Match(input, pattern);  
if (match.Success)  
{  
    Console.WriteLine($"Találat: {match.Value}");  
}
```

Matches - összes egyezés megkeresése

```
MatchCollection matches = Regex.Matches(input, pattern);  
foreach (Match match in matches)  
{  
    Console.WriteLine($"Találat: {match.Value}");  
}
```

Replace - csere

```
string result = Regex.Replace(input, pattern, replacement);
```

Regex opciók

```
// Nagy-kisbetű érzéketlen  
bool isMatch = Regex.IsMatch(input, pattern, RegexOptions.IgnoreCase);  
  
// Több sor kezelése  
bool isMatch = Regex.IsMatch(input, pattern, RegexOptions.Multiline);  
  
// Kombinált opciók  
bool isMatch = Regex.IsMatch(input, pattern,  
    RegexOptions.IgnoreCase | RegexOptions.Multiline);
```

Regex karakterosztályok

```
\d    // Számjegy [0-9]
\w    // Szó karakter [a-zA-Z0-9_]
\s    // Whitespace karakter (szóköz, tab, újsor)
.     // Bármilyen karakter (újsor kivételével)
^     // Sor eleje
$     // Sor vége
+     // Egy vagy több
*     // Nulla vagy több
?     // Nulla vagy egy
{n}   // Pontosán n darab
{n,}  // Legalább n darab
{n,m} // n-től m-ig
```

Praktikus példák validálásra

```
public class Validator
{
    // Telefonszám (magyar formátum)
    public static bool IsValidPhoneNumber(string phone)
    {
        return Regex.IsMatch(phone, @"^(\+36|06)?[1-9][0-9]{7,8}$");
    }

    // Irányítószám
    public static bool IsValidPostalCode(string postalCode)
    {
        return Regex.IsMatch(postalCode, @"^\d{4}$");
    }

    // Csak betűk és szóközők
    public static bool IsValidName(string name)
    {
        return Regex.IsMatch(name, @"^[a-zA-ZáéíóőúűűűÁÉÍÓÖŐÚÚÚ\S]+$");
    }

    // URL validálás (egyszerű)
    public static bool IsValidUrl(string url)
    {
        return Regex.IsMatch(url, @"^https?:\/\/\.\.\.+$");
    }
}
```

Összefoglaló

ArgumentException

- **Mikor:** Érvénytelen paraméterek esetén
- **Hogyan:** `throw new ArgumentException("hibaüzenet")`

- **Hasznos:** Property validálásban, metódus paraméter ellenőrzésben

DateTime

- **Dátum számítás:** `DateTime.Now`, `AddYears()`, `AddDays()`
- **Összehasonlítás:** `>`, `<`, `==` operátorok
- **Időkülönbség:** `TimeSpan` a különbség számításához

Regex

- **Mintaillesztés:** `Regex.IsMatch(text, pattern)`
- **Validálás:** email, név, jelszó ellenőrzésére
- **Minták:** `\d` számok, `\w` szóképek, `^$` sor eleje/vége

Ezek a technikák a C# fejlesztés alapkövei, és különösen hasznosak model osztályok validálásában!